



SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE

Objectives

01 Static Recap

02 Static blocks & nested
classes

03 Immutable objects

04 Enums

05 Records

Recap: Static You Already Know

- A static variable is shared by every object of the class, not per object
- A static method can be called on the class itself, no object needed
- Today we will see two more places static shows up

Static Blocks

- A block of code that runs once, when the class is first loaded
- Used to set up a static variable that needs real logic, not just a fixed value
- Runs automatically
- Runs before `main()` and before any object of the class is created

A Static Block

```
class Config {  
    static int sumOfFirstTen;  
    static {  
        int sum = 0;  
        for (int i = 1; i <= 10; i++) {  
            sum += i;  
        }  
        sumOfFirstTen = sum;  
    }  
}
```

- The static { } block runs exactly once, automatically
- No method call needed, Java runs it for you, the first time Config is used
- Useful when a static variable needs a calculation, not just a simple value

Static Blocks Really Do Run Once

```
class Test {  
    static {  
        System.out.println("Static block");  
    }  
    Test() {  
        System.out.println("Constructor");  
    }  
    public static void main(String[] args) {  
        new Test();  
        new Test();  
    }  
}
```

- "Static block" prints once, then "Constructor" prints twice
- The static block runs the first time Test is used at all
- Every new Test() after that only re-runs the constructor

Static Nested Classes

- A class declared inside another class, marked static
- It doesn't need an object of the outer class to exist
- Used to group a small helper class that only makes sense next to its outer class

A Static Nested Class

```
class Car {  
    static class Engine {  
        int horsepower;  
    }  
}  
  
Car.Engine e = new Car.Engine();
```

- Engine lives inside Car, but is built without ever making a Car
- Access it as Car.Engine
- Keeps small, closely related classes organized together

Question for the class

Car.Engine doesn't need a Car object to exist. What does that tell you about what the word static means for a nested class, compared to what it means for a variable?

Working it through

- static means the same thing here as it always has: it belongs to Car itself, not to any one Car object
- You never need to build a Car to use Car.Engine
- Same static keyword, same core idea just applied to a class this time instead of a variable

Why Overusing static Is Discouraged

- A static field is shared by the whole program, one part changing it can quietly affect every other part
- Static methods can't be overridden, so they don't work with polymorphism the way instance methods do
- Rule of thumb: static is great for small, self-contained helpers — not a substitute for real objects

Quick Recap: final

```
final class Car { }           // cannot be inherited  
final int x = 5;              // cannot be reassigned  
final void display() { }     // cannot be overridden
```

- final means "locked"
- On a class: nobody can extend it
- On a variable: it can't be reassigned
- On a method: it can't be overridden
- Next we'll use final on fields to help build immutable objects

What Does "Immutable" Mean?

- An object whose data can never change after it's created
- Once built, every field stays exactly as it was set
- No setters, and no other way to modify it later

Why Immutability Matters

- An immutable object can be safely shared anywhere, nothing can change it behind your back
- Easier to reason about: what you see when you get it is what you'll always get
- Safe to hand to other parts of a program at once, without worrying someone else changes it

Question for the class

Which of these are immutable, and which are mutable: String, Integer, ArrayList, int[]?

Working it through

- Immutable: String and Integer
- Once created, neither can be changed; any "change" actually creates a new object
- Mutable: ArrayList, arrays (int[])
- You can add, remove, or overwrite their contents in place

Building an Immutable Class

```
final class Point {  
    private final int x;  
    private final int y;  
  
    Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    int getX() { return x; }  
    int getY() { return y; }  
}
```

- final class: nobody can extend Point and change how it behaves
- private final fields: set once in the constructor, never reassigned
- Only getters, no setters

Question for the class

A class has all private final fields and a constructor. One field is a List, and a public getter returns that List directly. Is this class actually immutable?

Working it through

- No, final only locks the reference itself; it says nothing about what the object it points to allows
- A List is mutable, so anyone holding the returned reference can still add() or remove() from it
- Immutability is a promise about the whole class's behavior, not just what its fields are declared as

Common mistake

```
final class Team {  
    private final List<String> players;  
    Team(List<String> players)  
    {  
        this.players = players;  
    }  
  
    List<String> getPlayers() {  
        return players; // hands out the real, live list!  
    }  
}
```

- How do we make this immutable?

Common mistake

```
final class Team {  
    private final List<String> players;  
    Team(List<String> players)  
    {  
        this.players = List.copyOf(players);  
    }  
  
    List<String> getPlayers() {  
        return players;  
    }  
}
```

- Through defensive copying

The Problem Enums Solve

- Imagine representing a day of the week as a number
- 0 for Monday, 1 for Tuesday...
- Nothing stops someone from passing in 9, or mixing up which number means which day
- We want a fixed, named set of values

What is an Enum

- A special type representing a fixed set of constant values
- Each constant is one of the only instances of that type that will ever exist
- Safer than plain numbers or text
- The compiler checks it for you

Declaring an Enum

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,  
    SUNDAY  
}  
  
Day today = Day.WEDNESDAY;
```

- Day.WEDNESDAY is one specific, unique constant
- Only these seven values can ever exist for the type Day
- Comparing them is simple: today == Day.WEDNESDAY

Enums Can Carry Data

```
enum Day {  
    MONDAY(false), SATURDAY(true), SUNDAY(true); // and so on  
    private final boolean isWeekend;  
    Day(boolean isWeekend) {  
        this.isWeekend = isWeekend;  
    }  
    boolean isWeekend() { return isWeekend; }  
}
```

- Each constant can carry its own data, set through a constructor
- That constructor runs once per constant, when the enum is first loaded
- This works because every constant secretly is an object of type Day

Useful Built-in Enum Methods

- `name()` — gives back the constant's name as text, e.g. "WEDNESDAY"
- `ordinal()` — gives back its position in the list, starting at 0
- `values()` — gives back every constant in the enum, in order

Enums in a switch

```
switch (today) {  
    case MONDAY -> System.out.println("Back to class :(");  
    case SATURDAY, SUNDAY -> System.out.println("Weekend!");  
    default -> System.out.println("Midweek");  
}
```

- No need to write Day.MONDAY inside the switch, just MONDAY
- The compiler already knows today is a Day, so it limits the valid options for you

The Problem Records Solve

- In the Point class, we explicitly declare the fields, constructor, getters.
- If needed, we also have to add appropriate implementations of `equals()`, `hashCode()`, and `toString()` methods
- That's a lot of repeated, predictable code for something this simple
- We want a shortcut for classes that are really just... data

Declaring a Record

```
record Point(int x, int y) { }
```

- One line replaces the entire class from the immutability slide earlier
- Java automatically generates the constructor, getters, equals(), hashCode(), and toString()

Using a Record

```
Point p = new Point(3, 4);  
  
p.x();           // 3, not getX()  
p.y();           // 4  
p.toString();    // "Point[x=3, y=4]"
```

- Accessors are named after the field itself: x(), not getX()
- toString() is generated for you, already nicely formatted
- Two points with the same x and y are equal() automatically

Records vs Normal Classes

Normal class	Record
Lots of boilerplate	Very little code
Constructor written by hand	Generated automatically
Getters written by hand	Generated automatically
equals() written by hand	Generated automatically
hashCode() written by hand	Generated automatically
toString() written by hand	Generated automatically
Can be mutable or immutable	Intended for immutable data

Records Are Immutable by Design

- Every field in a record is automatically final
- There are no generated setters, only getters
- This is exactly the immutable Point class from earlier, written in one line instead of ten

Question for the class

A record's fields are all final, with no setters generated. Does that automatically make it just as safe as the immutable Point class we built by hand earlier?

Working it through

- For simple fields like `int` or `String`, nothing about them can change after construction
- If a field is itself a mutable object (like an array), the record can't stop someone changing what's inside it
- Records guarantee the fields can't be reassigned, not that everything reachable from them is unchangeable

Thank You